
Learning Bayesian Networks

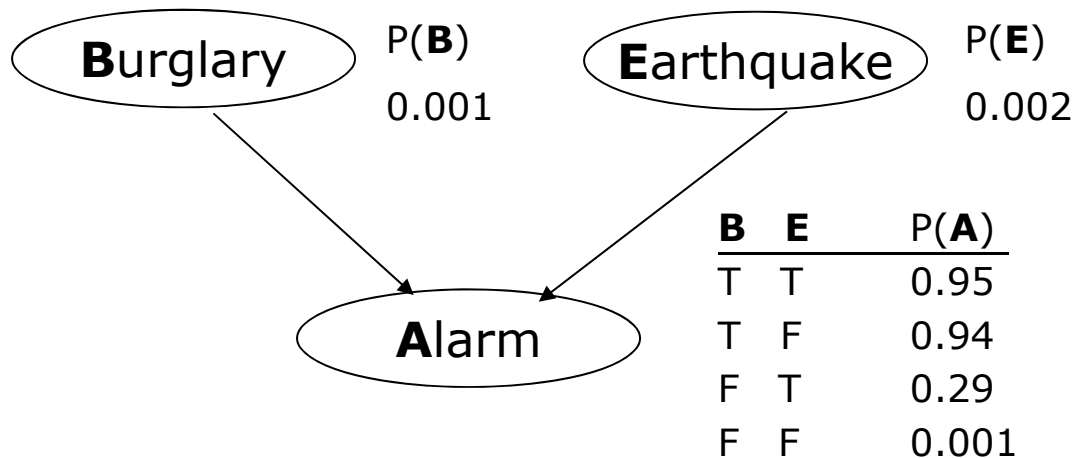
Nysret Musliu

Databases and Artificial Intelligence Group, TU Wien

Bayesian Networks

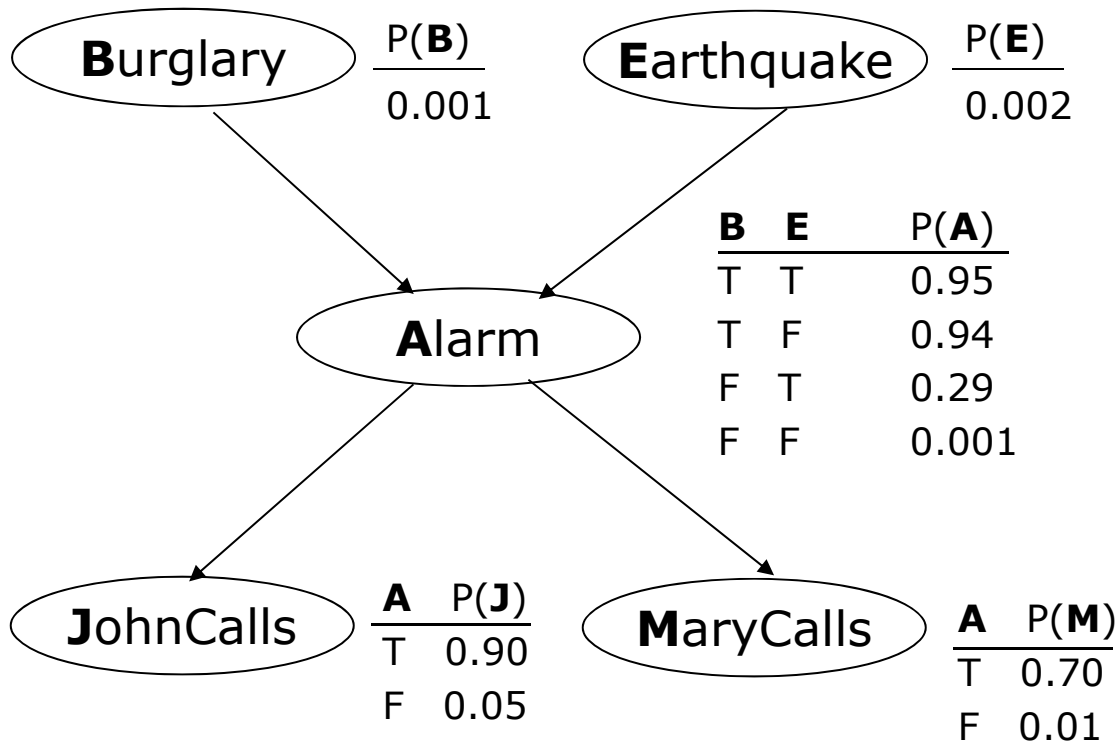
.....

- Set of variables
- Set of directed arcs between variables
- Acyclic graph
- Probabilities for every variable (node) given its parents



Bayesian Networks (Example)

.....



.....

Bayesian Networks

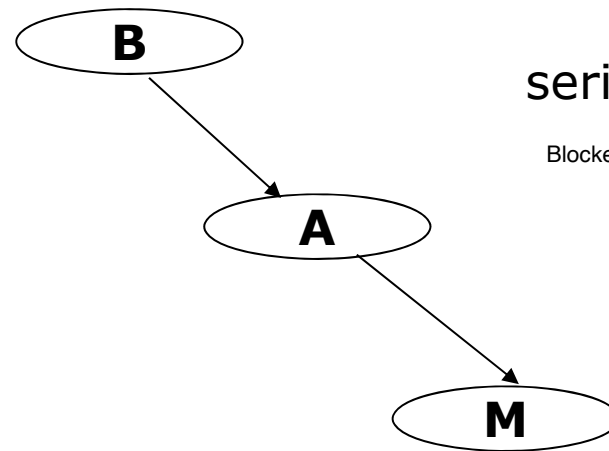
.....

- Components
 - Structure (how to determine it?)
 - Numerical parameters (probabilities)
- To answer questions we need to know joint probability distribution
- Example:
 - n variables (values 0 or 1):
 2^n number are needed to specify joint probability distribution
- Exploit independence between variables
 - D-separation

.....

D-separation

.....



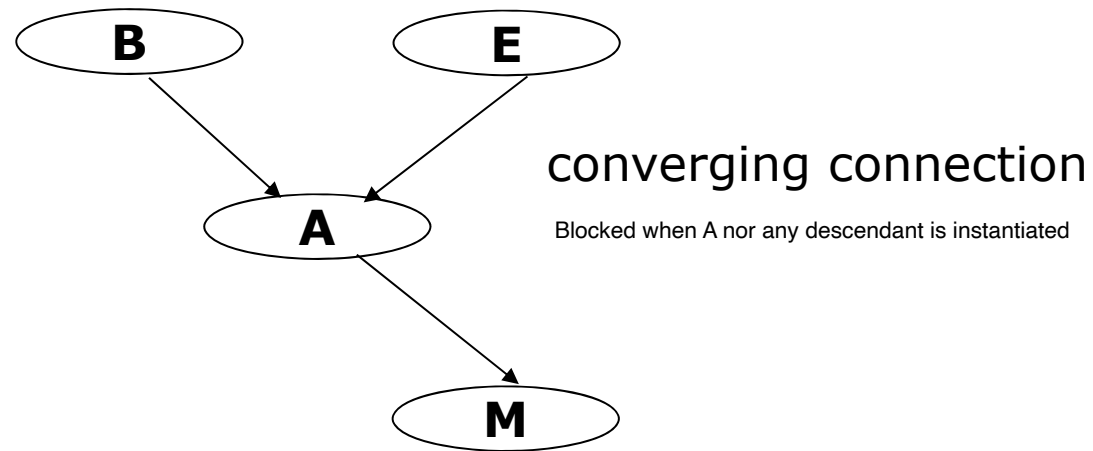
serial connection

Blocked when A is known, connected otherwise

.....

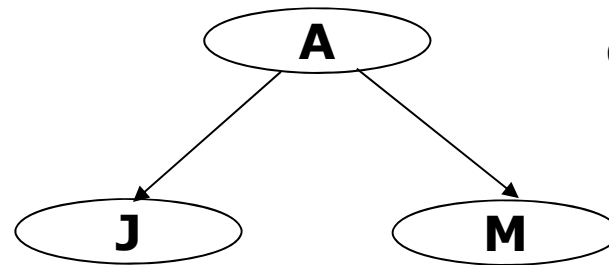
D-separation

.....



D-separation

.....



diverging connection

Blocked when A is known, connected otherwise

.....

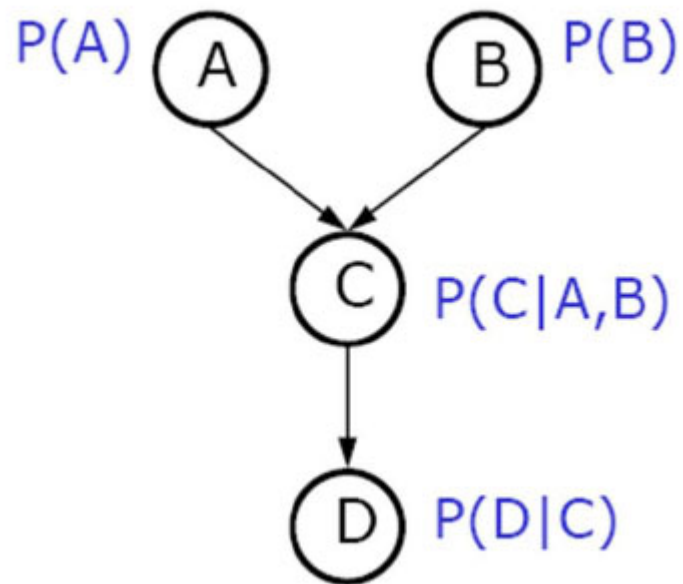
Chain Rule (Example)

.....

$$P(ABCD)=P(D|ABC) * P(ABC) =$$

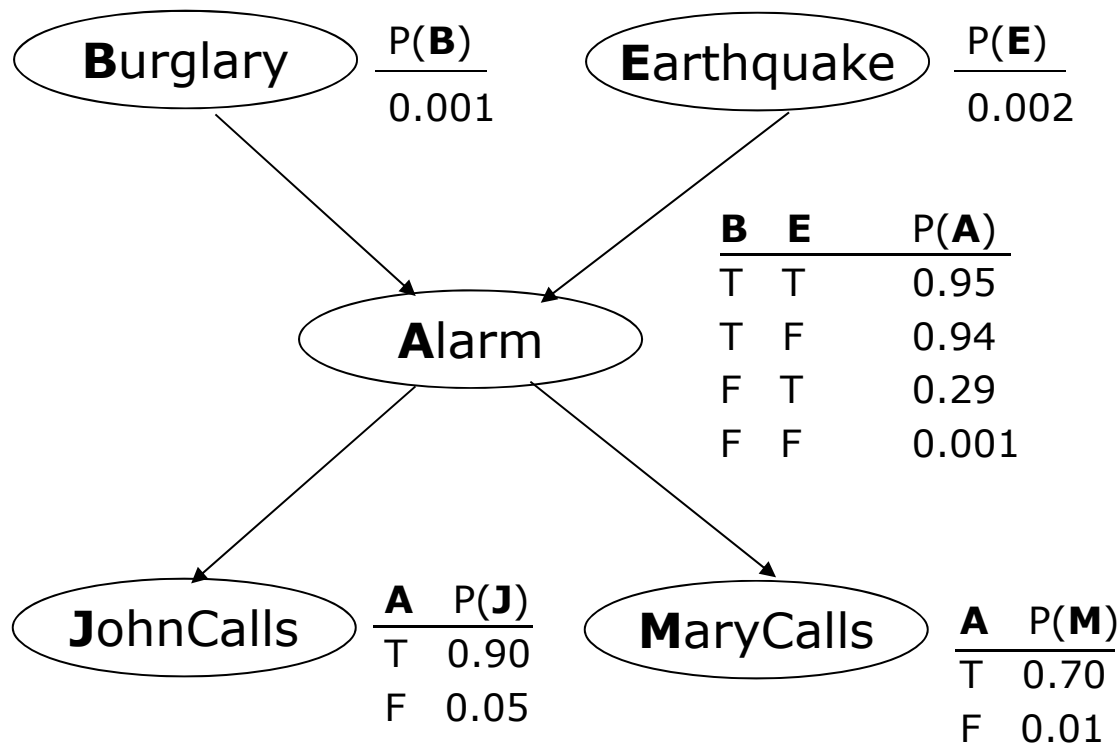
$$P(D|C) * P(C|AB) * P (AB)=$$

$$P(D|C) * P(C|AB) * P (A) * P(B)$$



Inference in Bayesian Networks

.....



Inference: answer questions based on some evidence

e.g.:

Evidence: JohnCalls

Query: probability of Burglary?

.....

Types of Questions

.....

Given some evidence, compute the probability distribution over query variables

$$P(Q_1, Q_2, \dots | E_1=e_1, E_2=e_2, \dots)$$

Maximum a posteriori probability (most likely explanation)

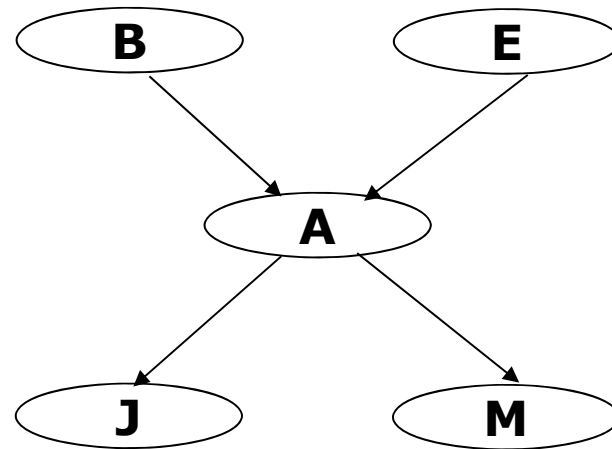
$$\operatorname{argmax}_q P(Q_1=q_1, Q_2=q_2, \dots | E_1=e_1, E_2=e_2, \dots)$$

.....

Answering Queries with Enumeration

.....

Sum over variables not involved in the query



$$P(B=t|J=t)=P(B=t, J=t)/P(J=t)$$

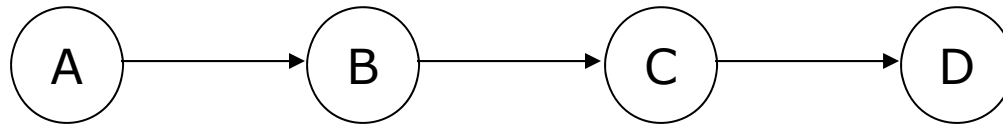
$$P(B=t, J=t)=\sum_{a \in \text{dom}(A)} \sum_e \sum_m P(J=t|A)*P(M|A)*P(A|B=t, E)*P(B=t)*P(E)$$

$$P(J=t)=\sum_a \sum_e \sum_m \sum_b P(J=t|A)*P(M|A)*P(A|B, E)*P(B)*P(E)$$

.....

Example 2

.....



$$P(D=t) = \sum_a \sum_b \sum_c P(D=t|C) * P(C|B) * P(B|A) * P(A)$$

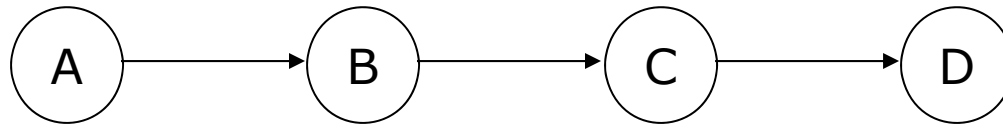
Too many computations: For all combinations of possible values of A, B, and C the factors should be multiplied

Practical domains include large number of variables

.....

Improving Efficiency

.....



$$P(D=t) = \sum_a \sum_b \sum_c P(D=t|C) * P(C|B) * P(B|A) * P(A)$$

$$P(D=t) = \sum_c P(D=t|C) \sum_b P(C|B) \sum_a P(B|A) * P(A)$$

Store intermediate results for $\sum_a P(B|A) * P(A)$ for each value of B in

$f_1(b)$:

$$f_1(b_1) = \sum_a P(B=b_1|A) * P(A)$$

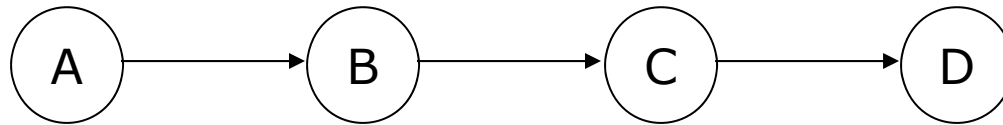
$$f_1(b_2) = \sum_a P(B=b_2|A) * P(A)$$

...

.....

Improving Efficiency

.....



$$P(D=t) = \sum_c P(D=t|C) \sum_b P(C|B) f_1(b) = \sum_c P(D=t|C) f_2(c)$$

$f_2(c)$:

$$f_2(c_1) = \sum_b P(C=c_1|B) f_1(b)$$

$$f_2(c_2) = \sum_b P(C=c_1|B) f_1(b)$$

...

.....

Variable Elimination Algorithm

.....

The previous algorithm is called Variable Elimination Algorithm

The computation can be performed more efficiently, if the order of variables is chosen carefully

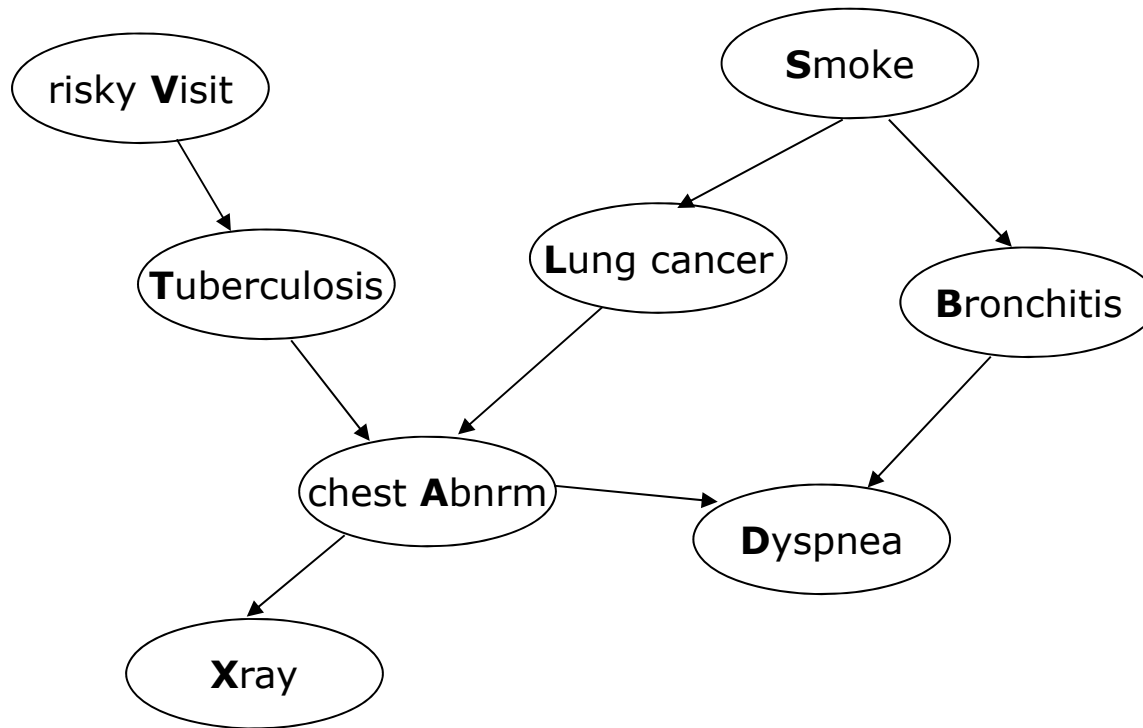
Finding best elimination order: NP-hard

Usually heuristics are used for finding good elimination orders

.....

Example 2

.....



$$P(D=t) = \sum_{abltsvx} P(D=t|A,B) P(B|S) P(L|S) P(S) P(V) P(T|V) P(X|A) P(A|T,L)$$

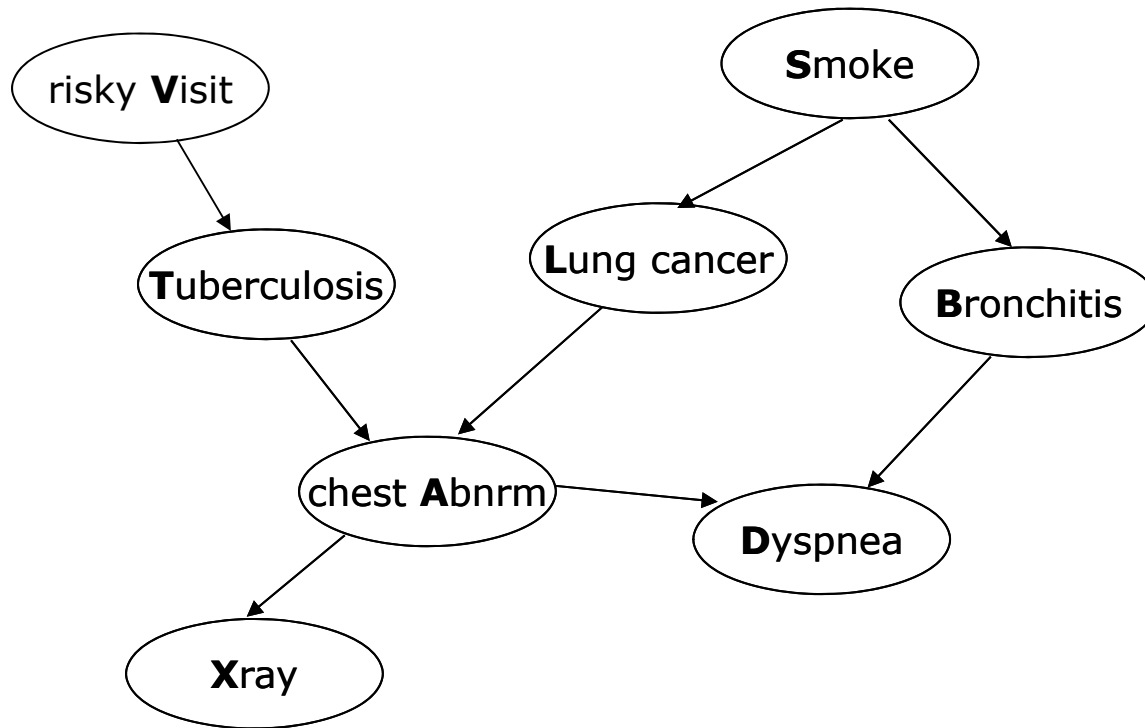
Example from Kaelbling, MIT (Lectures 15, 16)

<http://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-825-techniques-in-artificial-intelligence-sma-5504-fall-2002/lecture-notes/>

.....

Example 2

.....



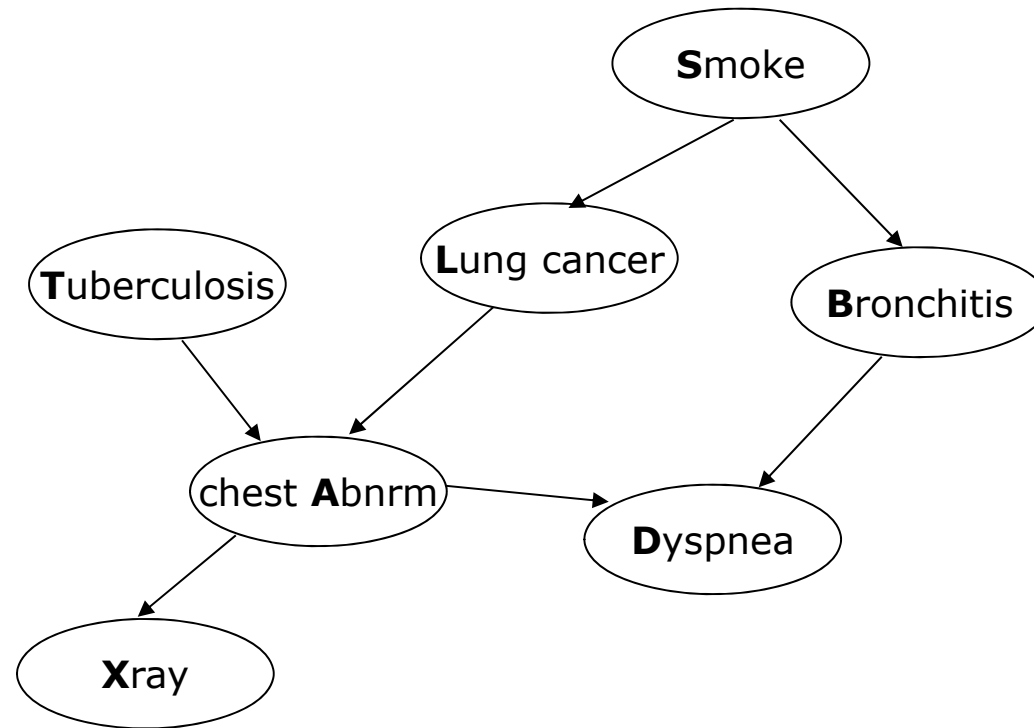
Eliminate V : $f_1(t) = \sum_v P(T|V) P(V)$

$P(D=t) = \sum_{abltsx} P(D=t|A,B) P(B|S) P(L|S) P(S) P(X|A) P(A|T,L) f_1(t)$

.....

Example 2

.....



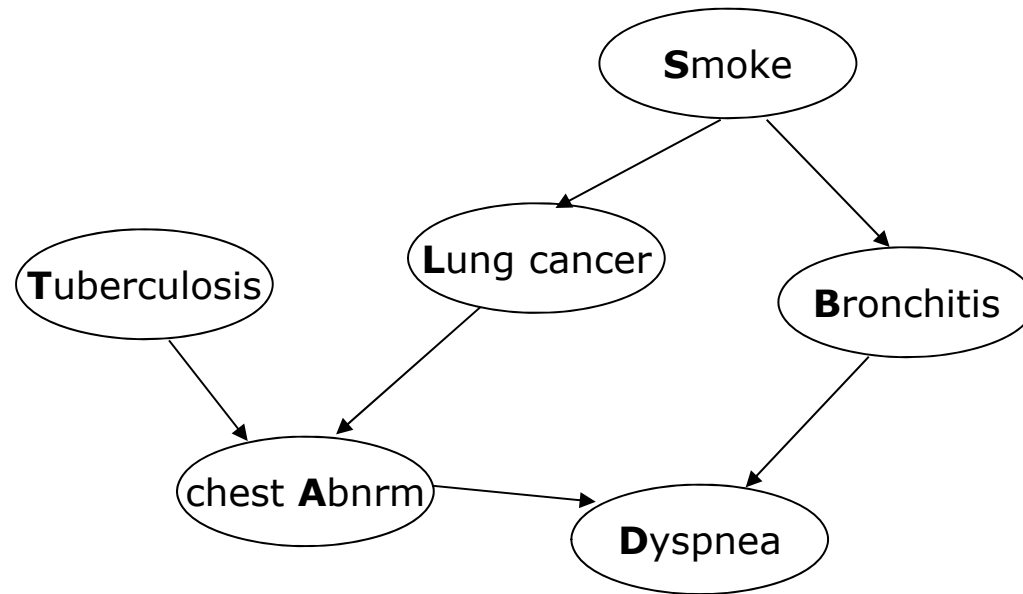
Eliminate X : $\sum_x P(X|A) = 1$

$$P(D=t) = \sum_{ablt_s} P(D=t|A,B) P(B|S) P(L|S) P(S) P(A|T,L) f_1(t)$$

.....

Example 2

.....



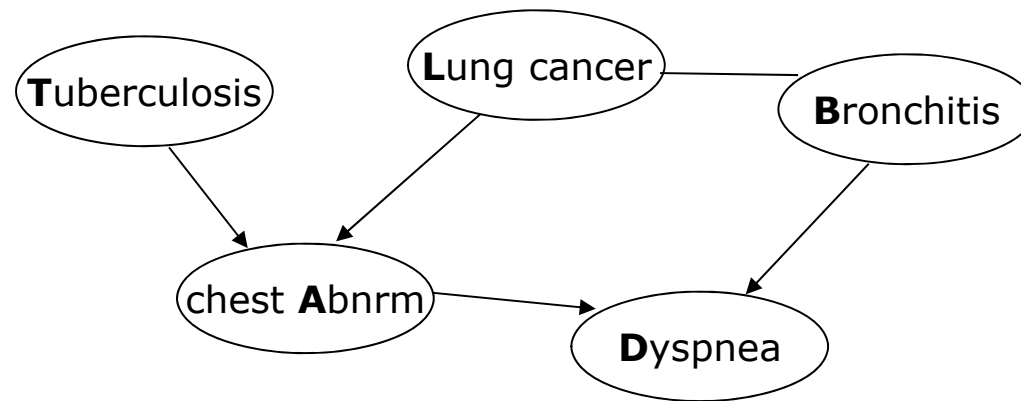
Eliminate S : $f_2(b,l) = \sum_s P(B|S) P(L|S) P(S)$

$P(D=t) = \sum_{a,b,l} P(D=t|A,B) P(A|T,L) f_2(b,l) f_1(t)$

.....

Example 2

.....



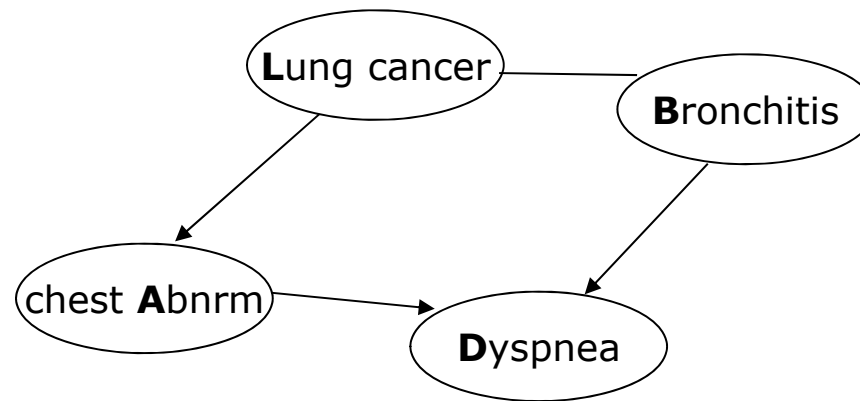
Eliminate T : $f_3(a,l) = \sum_t P(A|T,L) f_1(t)$

$P(D=t) = \sum_{abl} P(D=t|A,B) f_2(b,l) f_3(a,l)$

.....

Example 2

.....



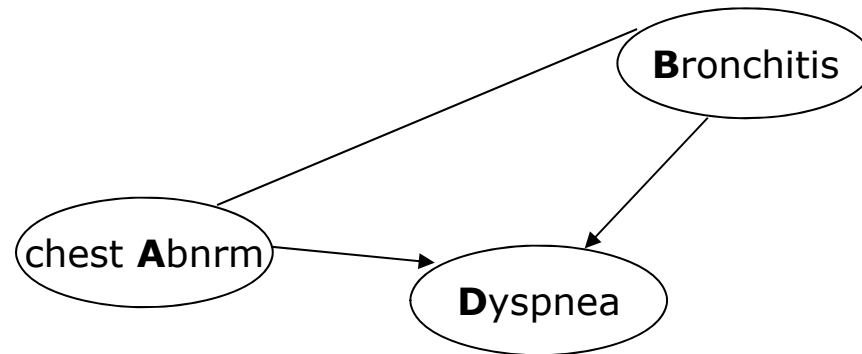
Eliminate L: $f_4(a,b) = \sum_l f_2(b,l) f_3(a,l)$

$P(D=t) = \sum_{ab} P(D=t|A,B) f_4(a,b)$

.....

Example 2

.....



Eliminate B: $f_5(a) = \sum_b P(D=t|A,B) f_4(a,b)$

$P(D=t) = \sum_a f_5(a)$



.....

Good Variable Elimination can improve the efficiency, but:

Exponential in size of largest factor

In large networks other methods which do approximation are used (e.g. Sampling)

.....

Learning Bayesian Networks

.....

Possibilities for building networks:

- Human experts
- Learning from data
- Combination of both approaches

Human experts are good on finding a structure

Computers better on calculating probabilities

.....

Learning from data

.....

Data are given:

e.g.

Data of patients and their and information if they had a particular disease

Structure known (for example from human expert):

Learn probabilities

Structure not known:

Learn the structure of the Bayesian network and the probabilities

Another case:

Not all variables are observable

Not subject of this course

.....

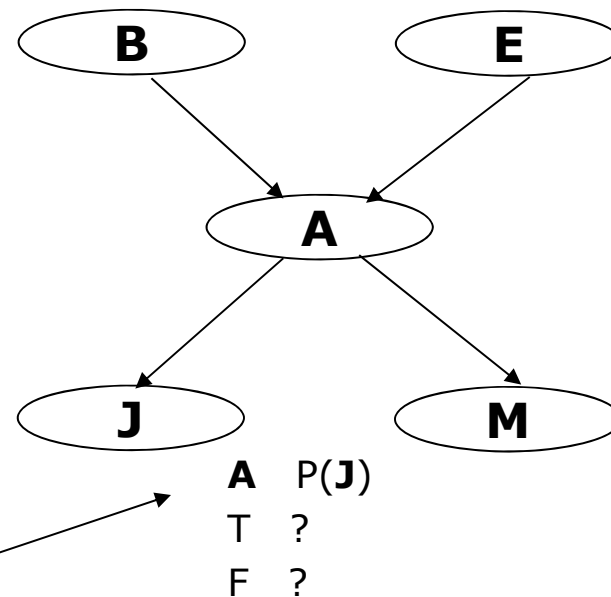
Learning BN with known structure

.....

Bayesian Network is given

Data set (D) is given

B	E	A	J	M
t	f	t	t	f
f	f	f	f	t
f	t	f	t	f
...				



Find model M (conditional probability tables - CPTs)

Maximum likelihood model
maximize $P(D|M)$

.....

Computing CPTs

.....

B	E	A	J	M
t	f	t	t	f
f	f	f	f	t
f	t	f	t	f
...				

Use counts

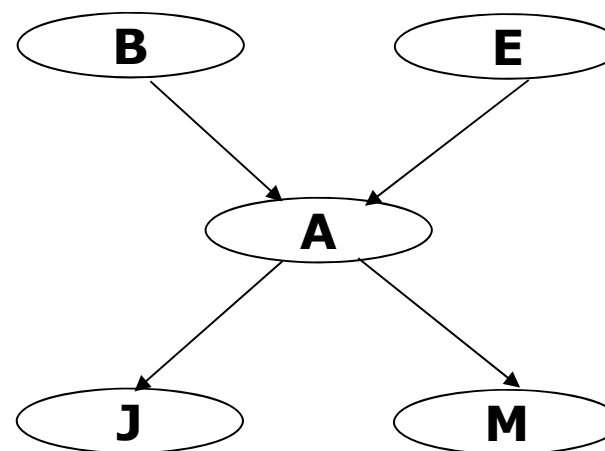
$$P(J|A) = (\#A = \text{true} \wedge J = \text{true}) / \#(A = \text{true})$$

Avoid 0 probabilities (Laplace correction):

$$P(J|A) = (\#(A = \text{true} \wedge J = \text{true}) + \mathbf{1}) / (\#(A = \text{true}) + \mathbf{2})$$

$$P(E) = (\#(E = \text{true}) + \mathbf{1}) / (k + \mathbf{2})$$

k: number of samples



P(E)

A **P(J)**
 T ?
 F ?

Laplace correction

.....

Goodness of Fit

Data set D

Model M

Goodness of the fit:

$$P(D|M) = \prod_j P(s^j|M) = \prod_j \prod_i P(N_i = v_i^j | \text{Parents}(N_i), M)$$

log likelihood

$$\log P(D|M) = \sum_j \sum_i \log P(N_i = v_i^j | \text{Parents}(N_i), M)$$

Learning the Structure

.....

Objective

Find a network that

- is a good fit to data
- has low complexity (less parameters)

Maximize:

$$\log P(D|M) - \alpha \#M$$

α -> parameter which indicates how important is to reduce the complexity of the network

.....

Search problem

.....

Search for a Bayesian network that maximizes the objective

Exhaustive search not practical (to many networks)

Use heuristic techniques

- local search

- population based techniques

- ...

.....

Local search for Bayesian Networks

.....

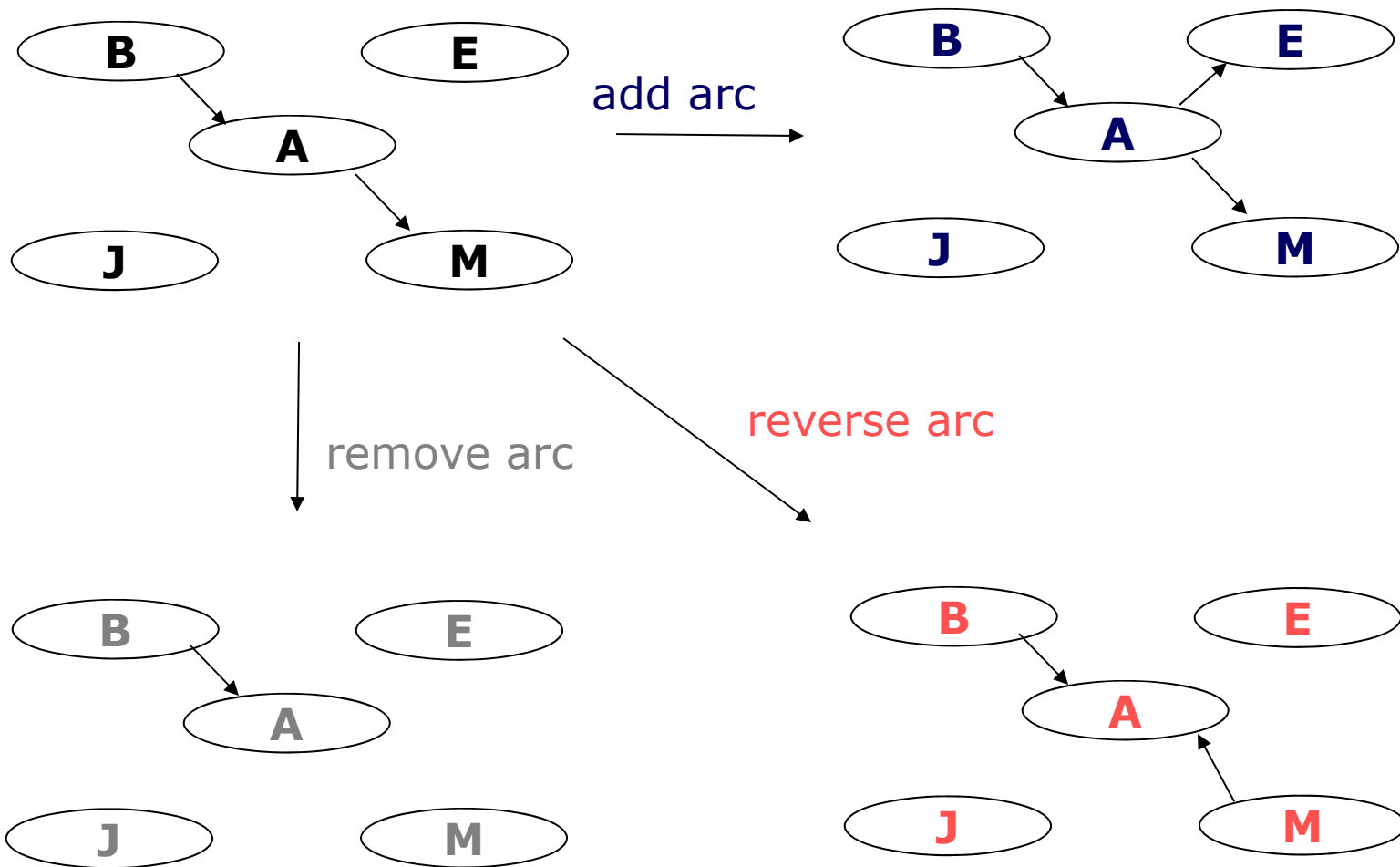
1. Construct an initial network
2. Calculate the score of the current BN network (with learned probabilities)
3. Generate the neighborhood by modifying the current network
4. Select one of networks in the neighborhood as a new current network for the next iteration
5. Go to Step 3 if termination criteria is not fulfilled

.....

Neighborhood relations

.....

current solution



Neighborhood exploration

.....

- Generate neighborhood solutions by applying neighborhood relations
- Hill Climbing, Tabu Search
 - all solutions in the neighborhood are generated
- Simulated Annealing
 - only one random solution in the neighborhood is generated

.....

Selection of the solution

.....

- Hill Climbing
 - Best solution in the neighborhood is selected
- Simulated Annealing
 - The randomly generated solution is accepted based on some probability that depends from the quality of the generated solution and temperature T
 - Worse solution can be accepted for the next iteration
- Tabu Search
 - Best solution is selected if is not tabu (otherwise the solution that is not tabu is selected)
 - Worse solution can be accepted for the next iteration

.....

Literature

.....

MIT lectures:

<http://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-825-techniques-in-artificial-intelligence-sma-5504-fall-2002/lecture-notes/>

- Artificial Intelligence: A Modern Approach (Third edition) by Stuart Russell and Peter Norvig (Chapter 14, 20)
- Bayesian Network Classifiers in Weka for Version 3-5-7. Remco R. Bouckaert

.....

.....

Appendix A

A short introduction to local search techniques

.....

Definition of search problem

.....

- Given a search space S together with its feasible part

$F \subseteq S$, find $x \in F$ such that

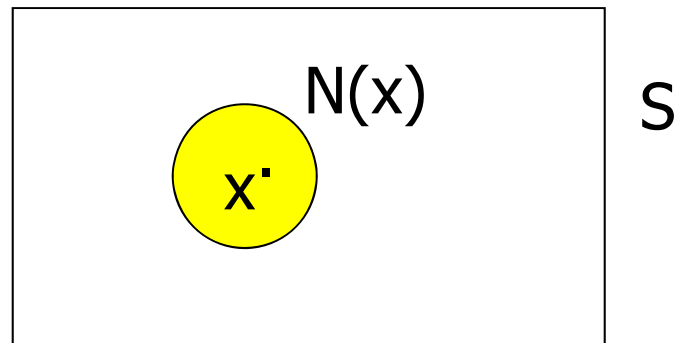
$$eval(x) \leq eval(y) \quad \text{for all } y \in F$$

- x that satisfies the above condition is called global optimum (for minimization problem)
-

Neighbourhood and local optima

.....

- Region of the search space that is near particular point in the space



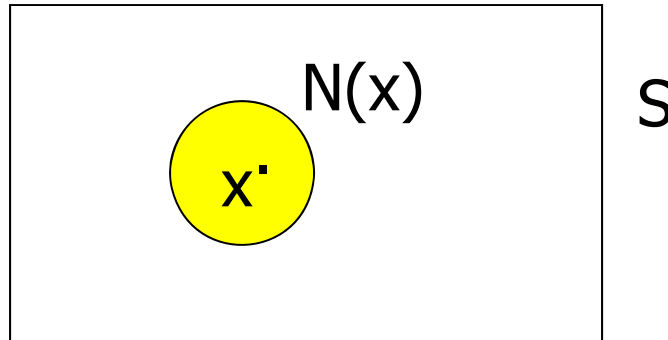
- A potential solution $x \in F$ is a local optimum with respect to the neighborhood N , if and only if
 $eval(x) \leq eval(y)$,
for all $y \in N(x)$

.....

Local Search Techniques

.....

- Are based on the neighbourhood of the current solution



- The solution is changed iteratively with so called neighbourhood relations (moves) until an acceptable or optimal solution is reached
-

Hill Climbing Algorithm

.....

1. Pick a random point in the search space
2. Consider all the neighbours of the current state
3. Choose the neighbour with the best quality and move to that state
4. Repeat 2 through 4 until all the neighbouring states are of lower quality
5. Return the current state as the solution state

.....

Simulated Annealing

.....
Prozedure simulated annealing

begin

$t=0$

Intialize T

select a current solution v_c at random

evaluate v_c

repeat

repeat

select a new solution v_n in the neighborhood of v_c

if $eval(v_c) < eval(v_n)$ **then** $v_c = v_n$

else if $random[0,1) < e^{\frac{eval(v_n) - eval(v_c)}{T}}$ **then** $v_c = v_n$

until (termination-condition)

$T = g(T, t)$

$t = t + 1$

until (halting-criterion)

end

Basic Tabu Search

.....

Procedure Tabu-Search

begin

 Initialize tabu list

 Generate randomly Initial Solution s_c

 Evaluate s_c

 repeat

 Generate all neighborhood solutions of the solution s_c

 Find best solution s_x in the neighborhood

 if s_x is not tabu solution then $s_c = s_x$

 else if 'aspiration criteria' is fulfilled then

$s_c = s_x$

 else

 find best not tabu solution in the neighborhood s_{nt}

$s_c = s_{nt}$

 Update tabu list

 until (terminate-condition)

end

.....

.....Simple Genetic Algorithm

```
{  
    initialize population;  
    evaluate population;  
    while TerminationCriteriaNotSatisfied  
    {  
        select parents for reproduction;  
        perform recombination and mutation;  
        evaluate population;  
    }  
}
```

Genetic Algorithms: A Tutorial, Wendy Williams

.....

.....

Appendix (D-Separation)

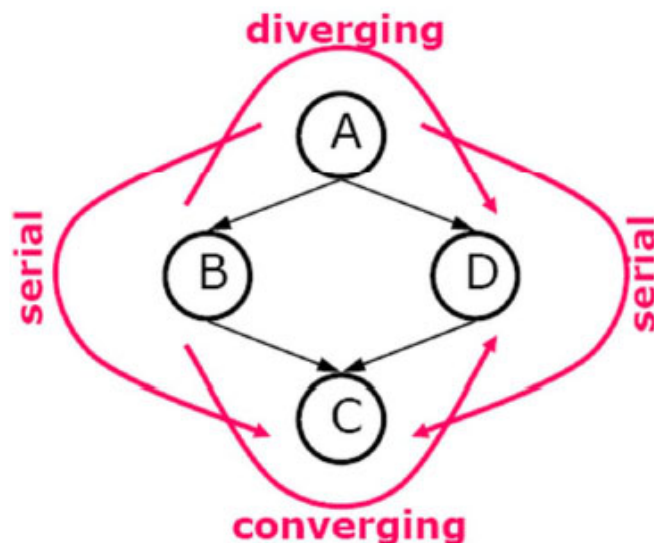
See lectures of Kaelbling, MIT (Lecture 15)

- <http://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-825-techniques-in-artificial-intelligence-sma-5504-fall-2002/lecture-notes/>

.....

D-separation

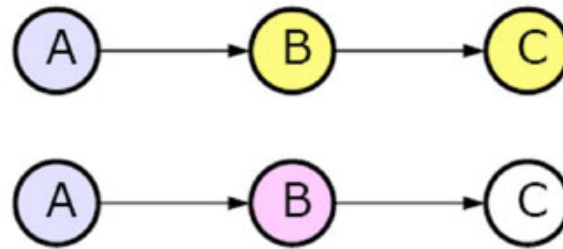
- Two variables A and B are **d-separated** iff for **every** path between them, there is an intermediate variable V such that either
 - The connection is serial or diverging and V is known
 - The connection is converging and neither V nor any descendant is instantiated
- Two variables are **d-connected** iff they are not d-separated



- A-B-C: serial, blocked when B is known, connected otherwise
- A-D-C: serial, blocked when D is known, connected otherwise
- B-A-D: diverging, blocked when A is known, connected otherwise
- B-C-D: converging, blocked when C has no evidence, connected otherwise

.....

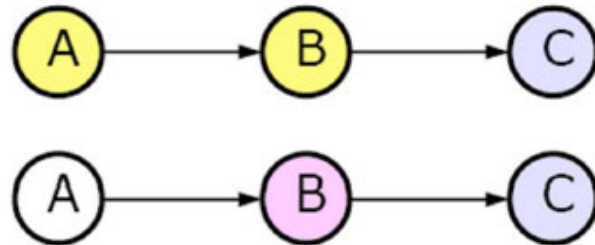
Forward Serial Connection



- Transmit evidence from A to C through unless B is instantiated (its truth value is known)
 - A = battery dead
 - B = car won't start
 - C = car won't move
- Knowing about A will tell us something about C
- But, if we know B, then knowing about A will not tell us anything new about C.

.....

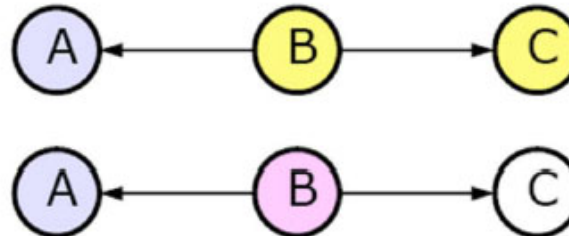
Backward Serial Connection



- Transmit evidence from C to A through unless B is instantiated (its truth value is known)
 - A = battery dead
 - B = car won't start
 - C = car won't move
- Knowing about C will tell us something about A
- But, if we know B, then knowing about C will not tell us anything new about A

.....

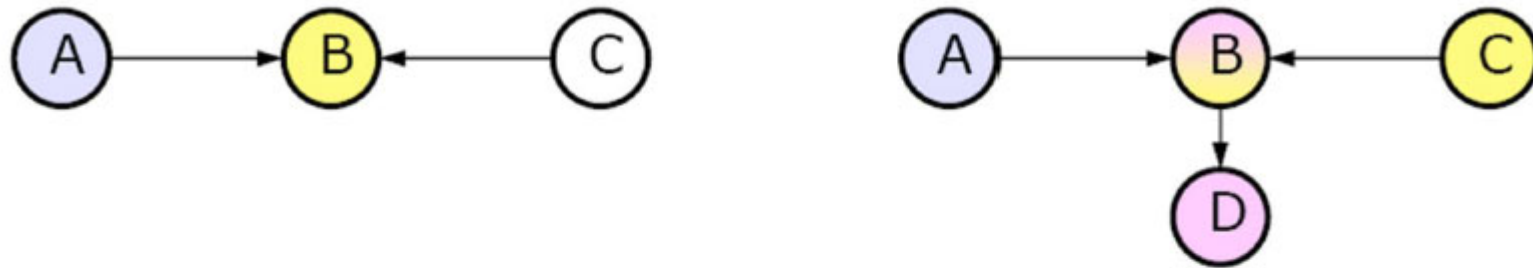
Diverging Connection



- Transmit evidence through B unless it is instantiated
 - A = Watson crash
 - B = Icy
 - C = Holmes crash
- Knowing about A will tell us something about C
- Knowing about C will tell us something about A
- But, if we know B, then knowing about A will not tell us anything new about C, or vice versa

.....

Converging Connection

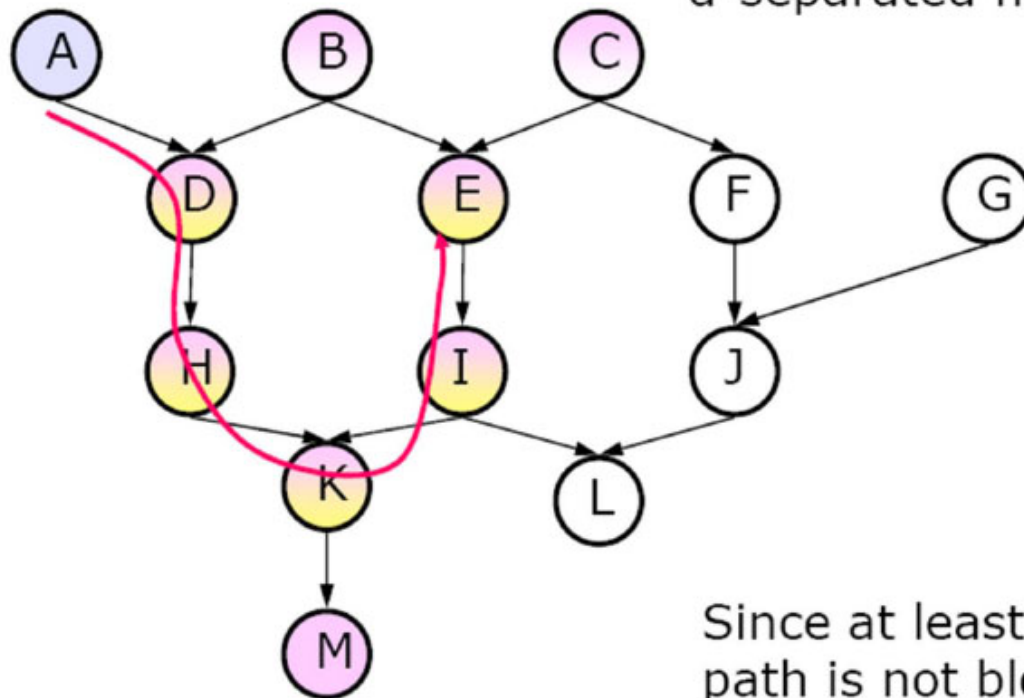


- Transmit evidence from A to C only if B or a descendant of B is instantiated
 - A = Bacterial infection
 - B = Sore throat
 - C = Viral Infection
- Without knowing B, finding A does not tell us anything about B
- If we see evidence for B, then A and C become dependent (potential for “explaining away”). If we find bacteria in patient with a sore throat, then viral infection is less likely.

....

D-Separation Example

Given M is known, is A d-separated from E?



Since at least one path is not blocked, A is not d-separated from E

.....